

The Complete Python Programming Guide

From Zero to Job-Ready — Language Mastery, Problem Solving, and Interview Preparation

Python is the world's most popular general-purpose programming language. It powers web backends (Instagram, Spotify), data science (Pandas, NumPy), machine learning (TensorFlow, PyTorch), automation, scripting, and rapid prototyping. This document is your complete reference.

PART I: PYTHON FUNDAMENTALS

Chapter 1: Why Python?

Python vs C/C++ — A Philosophical Difference

Aspect	C/C++	Python
Philosophy	Maximum control and performance	Maximum readability and productivity
Typing	Static (declare types at compile time)	Dynamic (types resolved at runtime)
Memory	Manual (<code>malloc/free</code>)	Automatic (garbage collector)
Speed	Very fast (compiled to machine code)	Slower (~10-100× than C for CPU work)
Syntax	Verbose, braces <code>{}</code> , semicolons <code>;</code>	Clean, indentation-based, minimal
Use Case	Embedded, OS, games, real-time	Web, data, ML, automation, scripting
Learning Curve	Steep (pointers, memory, segfaults)	Gentle (focus on logic, not plumbing)

Key Insight: Python is not a replacement for C. They serve different purposes. Many Python libraries (NumPy, TensorFlow) are actually written in C under the hood for performance, with Python providing the user-friendly interface.

Running Python

```
# Check version (need 3.8+)
python --version
python3 --version

# Run a script
python my_script.py

# Interactive REPL (Read-Eval-Print Loop)
python
>>> 2 + 2
4
>>> exit()
```

Hello World

Chapter 2: Variables, Types, and Operators

Variables

Numeric Types

2 / 40

```
# Arithmetic
10 / 3    # 3.3333... (true division, always returns float)
10 // 3   # 3         (floor division, returns int)
10 % 3    # 1         (modulo)
2 ** 10   # 1024      (exponentiation)

# Useful built-ins
abs(-42)   # 42
max(3, 7, 1) # 7
min(3, 7, 1) # 1
round(3.14159, 2) # 3.14
divmod(17, 5) # (3, 2) → quotient and remainder
```

Strings

Strings in Python are **immutable** — once created, they cannot be changed. Any operation that "modifies" a string actually creates a new one.

```
s = "Hello, World!"

# Indexing (0-based)
s[0]    # 'H'
s[-1]   # '!' (negative index = from the end)
s[-3]   # 'l'

# Slicing [start:stop:step] (stop is EXCLUSIVE)
s[0:5]   # 'Hello'
s[7:]    # 'World!'
s[:5]    # 'Hello'
s[::2]   # 'Hlo ol!' (every 2nd character)
s[::-1]  # '!dlroW ,olleH' (reversed!)

# String methods (returns NEW string, original unchanged)
s.lower()    # 'hello, world!'
s.upper()    # 'HELLO, WORLD!'
s.strip()    # Remove leading/trailing whitespace
s.split(", ") # ['Hello', 'World!']
s.replace("World", "Python") # 'Hello, Python!'
s.startswith("Hello") # True
s.endswith("!") # True
s.find("World") # 7 (index of first occurrence)
s.count("l") # 3
"42".isdigit() # True
"hello".isalpha() # True

# f-strings (formatted string literals – Python 3.6+)
name = "Dhruv"
age = 20
print(f"My name is {name} and I am {age} years old")
print(f"Next year I'll be {age + 1}")
```

```

print(f"Pi is approximately {3.14159:.2f}") # "Pi is approximately 3.14"
print(f"{'centered':^20}") # "          centered          "
print(f"{'left':<20}") # "left                        "
print(f"{'right':>20}") # "                        right"
print(f"0x{255:02X}") # "0xFF"

# Multi-line strings
text = """This is a
multi-line string.
It preserves line breaks."""

# Raw strings (ignore escape sequences – useful for regex and file paths)
path = r"C:\Users\dhruv\file.txt" # Backslashes are literal, not escapes

# String multiplication
"ha" * 3 # "hahaha"
"-" * 40 # "-----"

```

Type Conversion

```

int("42") # 42 (str → int)
float("3.14") # 3.14 (str → float)
str(42) # "42" (int → str)
int(3.99) # 3 (float → int, truncates!)
bool(0) # False (0, "", [], None → False; everything else → True)
list("hello") # ['h', 'e', 'l', 'l', 'o']

```

Chapter 3: Data Structures

Python has four built-in collection types. This is one of its greatest strengths.

Lists (Mutable, Ordered Sequence)

```

# Creation
nums = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True, [10, 20]] # Can mix types
empty = []

# Access
nums[0] # 1 (first element)
nums[-1] # 5 (last element)
nums[1:3] # [2, 3] (slice)

# Modification (lists are MUTABLE)
nums[0] = 99 # [99, 2, 3, 4, 5]
nums.append(6) # [99, 2, 3, 4, 5, 6] – add to end
nums.insert(1, 100) # [99, 100, 2, 3, 4, 5, 6] – insert at index
nums.extend([7, 8]) # [..., 5, 6, 7, 8] – add multiple

```

```

nums.pop()           # Remove and return last element (8)
nums.pop(0)          # Remove and return element at index 0 (99)
nums.remove(3)        # Remove first occurrence of value 3
del nums[1]           # Delete element at index 1
nums.clear()          # Remove all elements

# Searching
nums = [5, 3, 8, 1, 9, 3]
3 in nums             # True (membership test – O(n))
nums.index(8)         # 2 (index of first occurrence)
nums.count(3)         # 2 (how many times 3 appears)

# Sorting
nums.sort()           # In-place sort: [1, 3, 3, 5, 8, 9]
nums.sort(reverse=True) # Descending: [9, 8, 5, 3, 3, 1]
sorted_nums = sorted(nums) # Returns NEW sorted list (original unchanged)

# Other
len(nums)             # 6
sum(nums)             # 29
min(nums), max(nums)  # 1, 9
nums.reverse()        # Reverse in-place
nums.copy()           # Shallow copy

# List comprehension (Pythonic way to create lists)
squares = [x**2 for x in range(10)] # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
evens = [x for x in range(20) if x % 2 == 0] # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
pairs = [(x, y) for x in range(3) for y in range(3)] # All (x,y) combinations

# Nested list comprehension
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row] # [1, 2, 3, 4, 5, 6, 7, 8, 9]

```

Tuples (Immutable, Ordered Sequence)

```

# Tuples are like lists but CANNOT be modified after creation
point = (3, 4)
rgb = (255, 128, 0)
single = (42,) # Note the comma! (42) is just an int in parentheses

# Access (same as lists)
point[0] # 3
point[-1] # 4

# Cannot modify
point[0] = 10 # TypeError! Tuples are immutable.

# Tuple unpacking (extremely useful)
x, y = point # x=3, y=4

```

```

a, b, c = rgb          # a=255, b=128, c=0
first, *rest = [1, 2, 3, 4, 5] # first=1, rest=[2, 3, 4, 5]

# Swap variables (Pythonic – no temp variable needed!)
a, b = b, a

# Why tuples?
# 1. Safer: can't accidentally modify
# 2. Can be used as dictionary keys (lists can't)
# 3. Slightly faster than lists
# 4. Signal intent: "this data should not change"

```

Dictionaries (Mutable, Key-Value Mapping)

```

# Creation
student = {
    "name": "Dhruv",
    "age": 20,
    "courses": ["Physics", "CS", "Math"]
}
empty_dict = {}

# Access
student["name"]          # "Dhruv"
student.get("name")      # "Dhruv" (same, but won't crash if key missing)
student.get("gpa", 0.0)  # 0.0 (returns default if key doesn't exist)
student["gpa"]           # KeyError! Use .get() when unsure

# Modification
student["gpa"] = 3.8      # Add new key-value pair
student["age"] = 21       # Update existing value
del student["courses"]   # Delete a key-value pair
student.pop("gpa")        # Remove and return value
student.update({"city": "Mumbai", "age": 22}) # Merge another dict

# Iteration
for key in student:
    print(key, student[key])

for key, value in student.items():
    print(f"{key}: {value}")

for key in student.keys():
    print(key)

for value in student.values():
    print(value)

# Check membership
"name" in student        # True (checks KEYS, not values)

```

```
# Dict comprehension
squares = {x: x**2 for x in range(6)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Useful patterns
word = "abracadabra"
freq = {}
for char in word:
    freq[char] = freq.get(char, 0) + 1
# {'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}

# Or use Counter (from collections)
from collections import Counter
freq = Counter(word) # Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
```

Sets (Mutable, Unordered, Unique Elements)

```
# Creation
colors = {"red", "green", "blue"}
nums = set([1, 2, 2, 3, 3, 3]) # {1, 2, 3} – duplicates removed

# Operations
colors.add("yellow") # Add element
colors.remove("red") # Remove (KeyError if not found)
colors.discard("red") # Remove (no error if not found)

# Set operations (like math sets)
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

a | b # Union: {1, 2, 3, 4, 5, 6}
a & b # Intersection: {3, 4}
a - b # Difference: {1, 2} (in a but not in b)
b - a # {5, 6}
a ^ b # Symmetric difference: {1, 2, 5, 6} (in one but not both)
a <= b # Subset check: False
{3, 4} <= a # True ({3,4} is a subset of a)

# Membership test is O(1) – MUCH faster than lists!
42 in my_set # O(1) average
42 in my_list # O(n)

# Set comprehension
even_squares = {x**2 for x in range(10) if x % 2 == 0}
# {0, 4, 16, 36, 64}
```

Choosing the Right Data Structure

Need	Use	Why
Ordered collection, need to modify	<code>list</code>	Dynamic array, O(1) append
Ordered collection, don't modify	<code>tuple</code>	Immutable, hashable, safe
Key-value lookup	<code>dict</code>	O(1) average lookup
Unique elements, fast membership	<code>set</code>	O(1) average lookup, no duplicates
FIFO queue	<code>collections.deque</code>	O(1) append and popleft
Priority queue	<code>heapq</code>	O(log n) push and pop
Ordered dict	<code>dict</code> (Python 3.7+)	Insertion order preserved
Default values in dict	<code>collections.defaultdict</code>	Auto-creates missing keys
Named fields	<code>collections.namedtuple</code> or <code>dataclass</code>	Readable, lightweight

Chapter 4: Control Flow

Conditionals

```
# if / elif / else
temperature = 75

if temperature > 100:
    print("Boiling!")
elif temperature > 80:
    print("Hot")
elif temperature > 60:
    print("Warm")
else:
    print("Cool")

# Ternary expression (one-liner)
status = "adult" if age >= 18 else "minor"

# Truthy and Falsy values
# FALSY: None, False, 0, 0.0, "", [], {}, set(), ()
# TRUTHY: Everything else

if my_list:          # Pythonic check for non-empty list
    process(my_list)

if not my_string:    # Pythonic check for empty string
    print("String is empty")
```

Loops

```
# for loop (iterates over any iterable)
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 10, 3):   # 2, 5, 8 (start, stop, step)
    print(i)

for char in "Hello":       # H, e, l, l, o
    print(char)

for item in [10, 20, 30]:   # Iterate list
    print(item)

# enumerate: get both index AND value
fruits = ["apple", "banana", "cherry"]
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
# 0: apple
# 1: banana
# 2: cherry

# zip: iterate multiple sequences together
names = ["Alice", "Bob", "Charlie"]
scores = [95, 87, 92]
for name, score in zip(names, scores):
    print(f"{name}: {score}")

# while loop
count = 0
while count < 5:
    print(count)
    count += 1

# break and continue (same as C)
for i in range(100):
    if i == 5:
        break          # Exit loop entirely
    if i % 2 == 0:
        continue       # Skip this iteration

# for/else (unique to Python!)
for num in range(2, 10):
    for i in range(2, num):
        if num % i == 0:
            break
    else:
        # This runs if the inner loop did NOT break (i.e., num is prime)
        print(f"{num} is prime")
```

Chapter 5: Functions

```
# Basic function
def greet(name):
    """Greet a person by name.""" # Docstring (documentation)
    return f"Hello, {name}!"

result = greet("Dhruv") # "Hello, Dhruv!"

# Default parameters
def power(base, exponent=2):
    return base ** exponent

power(3)      # 9 (exponent defaults to 2)
power(3, 4)   # 81

# Keyword arguments (order doesn't matter)
power(exponent=3, base=2) # 8

# *args: Variable positional arguments (tuple)
def add_all(*args):
    return sum(args)

add_all(1, 2, 3, 4, 5) # 15

# **kwargs: Variable keyword arguments (dict)
def build_profile(**kwargs):
    return kwargs

build_profile(name="Dhruv", age=20, city="Mumbai")
# {'name': 'Dhruv', 'age': 20, 'city': 'Mumbai'}

# Combined
def func(a, b, *args, key="default", **kwargs):
    print(a, b, args, key, kwargs)

# Multiple return values (actually returns a tuple)
def divide(a, b):
    quotient = a // b
    remainder = a % b
    return quotient, remainder

q, r = divide(17, 5) # q=3, r=2

# Lambda (anonymous function)
square = lambda x: x ** 2
square(5) # 25

# Commonly used with map, filter, sorted
nums = [3, 1, 4, 1, 5, 9, 2, 6]
sorted(nums, key=lambda x: -x) # [9, 6, 5, 4, 3, 2, 1, 1]
```

```
list(filter(lambda x: x > 3, nums)) # [4, 5, 9, 6]
list(map(lambda x: x**2, nums))     # [9, 1, 16, 1, 25, 81, 4, 36]
```

Scope and Closures

```
# LEGB Rule: Local → Enclosing → Global → Built-in
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)    # "local"
    inner()
    print(x)        # "enclosing"

outer()
print(x)            # "global"

# Closure: inner function remembers outer function's variables
def make_multiplier(factor):
    def multiply(x):
        return x * factor # 'factor' is captured from outer scope
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
double(5)    # 10
triple(5)    # 15
```

Decorators

A decorator wraps a function to add behavior without modifying the original:

```
import time

def timer(func):
    """Decorator that measures execution time."""
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - start
        print(f"{func.__name__} took {elapsed:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)
```

```
    return "done"

slow_function() # Prints: "slow_function took 1.0012 seconds"

# Common built-in decorators
class MyClass:
    @staticmethod          # No self parameter, can't access instance
    def utility():
        pass

    @classmethod           # First param is the class (cls), not instance
    def create(cls):
        return cls()

    @property              # Access method like an attribute (no parentheses)
    def name(self):
        return self._name
```

Chapter 6: Object-Oriented Programming

Classes

```
class Dog:
    # Class variable (shared by ALL instances)
    species = "Canis familiaris"

    # Constructor (initializer)
    def __init__(self, name, age):
        # Instance variables (unique to each instance)
        self.name = name
        self.age = age
        self._tricks = []      # Convention: _ prefix = "private" (not enforced)

    # Instance method
    def bark(self):
        return f"{self.name} says Woof!"

    def teach_trick(self, trick):
        self._tricks.append(trick)

    # String representation
    def __repr__(self):
        return f"Dog(name='{self.name}', age={self.age})"

    def __str__(self):
        return f"{self.name}, {self.age} years old"

    # Comparison
    def __eq__(self, other):
        return self.name == other.name and self.age == other.age
```

```

    def __lt__(self, other):
        return self.age < other.age

# Usage
buddy = Dog("Buddy", 5)
max_dog = Dog("Max", 3)

print(buddy.bark())    # "Buddy says Woof!"
print(buddy)           # "Buddy, 5 years old" (calls __str__)
print(repr(buddy))     # "Dog(name='Buddy', age=5)" (calls __repr__)
print(buddy > max_dog) # True (5 > 3, uses __lt__)

# All dogs are the same species
print(buddy.species)   # "Canis familiaris"
print(Dog.species)     # "Canis familiaris"

```

Inheritance

```

class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def speak(self):
        return f"{self.name} says {self.sound}!"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "Woof") # Call parent constructor
        self.breed = breed

    def fetch(self, item):
        return f"{self.name} fetches the {item}!"

class Cat(Animal):
    def __init__(self, name):
        super().__init__(name, "Meow")

    def purr(self):
        return f"{self.name} purrs..."

# Polymorphism
animals = [Dog("Buddy", "Golden Retriever"), Cat("Whiskers"), Dog("Max",
"Poodle")]
for animal in animals:
    print(animal.speak()) # Each calls its own version

# isinstance and issubclass
isinstance(buddy, Dog)   # True

```

```

isinstance(buddy, Animal)  # True (Dog IS an Animal)
issubclass(Dog, Animal)    # True

```

Dataclasses (Python 3.7+ — Modern Way)

```

from dataclasses import dataclass, field

@dataclass
class Point:
    x: float
    y: float

    def distance_to(self, other):
        return ((self.x - other.x)**2 + (self.y - other.y)**2) ** 0.5

# Auto-generates: __init__, __repr__, __eq__
p1 = Point(3.0, 4.0)
p2 = Point(0.0, 0.0)
print(p1)                # Point(x=3.0, y=4.0)
print(p1.distance_to(p2)) # 5.0
print(p1 == Point(3.0, 4.0)) # True (auto __eq__)

@dataclass
class Student:
    name: str
    age: int
    grades: list = field(default_factory=list) # Mutable default
    gpa: float = 0.0                          # Simple default

s = Student("Dhruv", 20)
s.grades.append(95)

```

Magic/Dunder Methods (Complete Reference)

Method	Triggered By	Purpose
<code>__init__</code>	<code>MyClass()</code>	Constructor
<code>__del__</code>	Object garbage collected	Destructor
<code>__repr__</code>	<code>repr(obj)</code> , debugger	Developer-friendly string
<code>__str__</code>	<code>str(obj)</code> , <code>print(obj)</code>	User-friendly string
<code>__len__</code>	<code>len(obj)</code>	Length
<code>__getitem__</code>	<code>obj[key]</code>	Index access
<code>__setitem__</code>	<code>obj[key] = val</code>	Index assignment
<code>__contains__</code>	<code>item in obj</code>	Membership test

Method	Triggered By	Purpose
<code>__iter__</code>	<code>for x in obj</code>	Iteration
<code>__next__</code>	<code>next(obj)</code>	Next item in iterator
<code>__add__</code>	<code>obj + other</code>	Addition operator
<code>__sub__</code>	<code>obj - other</code>	Subtraction operator
<code>__mul__</code>	<code>obj * other</code>	Multiplication operator
<code>__eq__</code>	<code>obj == other</code>	Equality check
<code>__lt__</code>	<code>obj < other</code>	Less than
<code>__hash__</code>	<code>hash(obj)</code> , dict keys	Hashability
<code>__call__</code>	<code>obj()</code>	Make instance callable
<code>__enter__</code> / <code>__exit__</code>	<code>with obj:</code>	Context manager
<code>__bool__</code>	<code>bool(obj)</code> , <code>if obj:</code>	Truthiness

Chapter 7: Error Handling

```
# try / except / else / finally
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
except (ValueError, TypeError) as e:
    print(f"Error: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
else:
    # Runs ONLY if no exception occurred
    print(f"Result: {result}")
finally:
    # ALWAYS runs, even if exception occurred
    print("Cleanup complete")

# Raising exceptions
def set_age(age):
    if age < 0:
        raise ValueError(f"Age cannot be negative: {age}")
    if not isinstance(age, int):
        raise TypeError("Age must be an integer")
    return age

# Custom exceptions
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
```

```

        self.amount = amount
        super().__init__(f"Cannot withdraw ${amount}. Balance: ${balance}")

class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount

# Context managers (automatic cleanup with 'with')
# Files, database connections, locks – anything that needs cleanup
with open("data.txt", "r") as f:
    data = f.read()
# File is automatically closed here, even if an exception occurred inside

# Custom context manager
class Timer:
    def __enter__(self):
        self.start = time.time()
        return self
    def __exit__(self, *args):
        self.elapsed = time.time() - self.start
        print(f"Elapsed: {self.elapsed:.4f}s")

with Timer():
    # ... some code ...
    time.sleep(1)
# Prints: "Elapsed: 1.0012s"

```

Common Exception Types

Exception	When Raised
ValueError	Right type, wrong value: <code>int("abc")</code>
TypeError	Wrong type: <code>"hello" + 42</code>
IndexError	List index out of range: <code>lst[100]</code>
KeyError	Dict key doesn't exist: <code>d["missing"]</code>
AttributeError	Object doesn't have attribute: <code>"hello".nonexistent()</code>
NameError	Variable not defined: <code>print(undefined_var)</code>
FileNotFoundError	File doesn't exist: <code>open("missing.txt")</code>
ZeroDivisionError	Division by zero: <code>10 / 0</code>
ImportError	Module not found: <code>import nonexistent</code>

Exception	When Raised
<code>StopIteration</code>	Iterator exhausted
<code>RuntimeError</code>	Generic runtime error
<code>OverflowError</code>	Number too large (rare in Python)
<code>MemoryError</code>	Out of memory
<code>RecursionError</code>	Maximum recursion depth exceeded

PART II: INTERMEDIATE PYTHON

Chapter 8: Iterators and Generators

Iterators

```
# Any object with __iter__ and __next__ methods is an iterator
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

for num in Countdown(5):
    print(num)  # 5, 4, 3, 2, 1
```

Generators (Lazy Iterators)

Generators are functions that `yield` values one at a time instead of returning all at once. They use almost no memory regardless of how many values they produce.

```
# Generator function (uses yield instead of return)
def fibonacci():
    a, b = 0, 1
    while True:          # Infinite sequence!
        yield a          # Pause here, return value, resume on next call
        a, b = b, a + b

# Usage (only generates values as needed – lazy evaluation)
fib = fibonacci()
```

```

next(fib)    # 0
next(fib)    # 1
next(fib)    # 1
next(fib)    # 2
next(fib)    # 3

# Get first 10 Fibonacci numbers
from itertools import islice
list(islice(fibonacci(), 10)) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

# Generator expression (like list comprehension, but lazy)
squares_list = [x**2 for x in range(1000000)] # Creates entire list in memory!
squares_gen  = (x**2 for x in range(1000000)) # Creates nothing yet – O(1)
memory

sum(squares_gen) # Processes one value at a time, constant memory

# Practical example: process a huge file line by line
def read_large_file(filepath):
    with open(filepath, 'r') as f:
        for line in f:
            yield line.strip()

for line in read_large_file("huge_log.txt"):
    if "ERROR" in line:
        print(line)

```

Chapter 9: Modules and Packages

```

# Importing
import math
math.sqrt(16)    # 4.0
math.pi         # 3.14159...

from math import sqrt, pi    # Import specific names
sqrt(16)                  # No need for math. prefix

from math import *         # Import everything (AVOID in production code)

import math as m           # Alias
m.sqrt(16)

# Useful standard library modules
import os                 # File system operations
import sys                # System-specific parameters
import json               # JSON encoding/decoding
import csv                # CSV file reading/writing
import re                 # Regular expressions
import datetime            # Date and time
import random              # Random numbers
import hashlib             # Hashing (SHA256, MD5)

```

```
import pathlib      # Object-oriented file paths
import argparse     # Command-line argument parsing
import logging      # Logging
import unittest     # Unit testing
import collections  # Counter, defaultdict, deque, namedtuple
import itertools    # Combinatorics, infinite iterators
import functools    # Higher-order functions, caching
```

Creating Your Own Module

```
# my_utils.py
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

PI = 3.14159

# main.py
import my_utils
result = my_utils.add(3, 4)

# Package structure (directory with __init__.py)
# my_package/
#   __init__.py
#   module_a.py
#   module_b.py
#   sub_package/
#     __init__.py
#     module_c.py
```

Chapter 10: File I/O and Data Formats

```
# Reading a text file
with open("data.txt", "r") as f:
    content = f.read()          # Entire file as string
    # OR
    lines = f.readlines()      # List of lines
    # OR
    for line in f:              # Line by line (memory efficient)
        print(line.strip())

# Writing a text file
with open("output.txt", "w") as f: # "w" = overwrite, "a" = append
    f.write("Line 1\n")
    f.write("Line 2\n")
```

```
# JSON (JavaScript Object Notation)
import json

data = {"name": "Dhruv", "age": 20, "scores": [95, 87, 92]}

# Write JSON
with open("data.json", "w") as f:
    json.dump(data, f, indent=2) # Pretty-printed

# Read JSON
with open("data.json", "r") as f:
    loaded = json.load(f)

# Convert to/from string
json_str = json.dumps(data)
data_back = json.loads(json_str)

# CSV
import csv

# Write CSV
with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Age", "Grade"])
    writer.writerow(["Alice", 20, "A"])
    writer.writerow(["Bob", 22, "B"])

# Read CSV
with open("students.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["Name"], row["Grade"])

# pathlib (modern file path handling)
from pathlib import Path

p = Path("data") / "logs" / "app.log" # Cross-platform path joining
p.exists() # True/False
p.is_file() # True/False
p.suffix # ".log"
p.stem # "app"
p.parent # Path("data/logs")
p.read_text() # Read entire file
p.write_text("hi") # Write entire file

# List all .py files recursively
for py_file in Path(".").rglob("*.py"):
    print(py_file)
```

Chapter 11: Functional Programming Tools

```
# map: Apply function to every element
nums = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x * 2, nums)) # [2, 4, 6, 8, 10]

# filter: Keep elements that match condition
evens = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]

# reduce: Accumulate values
from functools import reduce
total = reduce(lambda acc, x: acc + x, nums) # 15
product = reduce(lambda acc, x: acc * x, nums) # 120

# any / all
nums = [2, 4, 6, 8]
all(x % 2 == 0 for x in nums) # True (all are even)
any(x > 5 for x in nums)      # True (at least one > 5)

# sorted with key
students = [("Alice", 3.9), ("Bob", 3.5), ("Charlie", 3.8)]
sorted(students, key=lambda s: s[1], reverse=True)
# [('Alice', 3.9), ('Charlie', 3.8), ('Bob', 3.5)]

# functools.lru_cache (memoization – cache function results)
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

fibonacci(100) # Instant! Without cache, this would take billions of years.
```

PART III: ADVANCED PYTHON

Chapter 12: Concurrency and Parallelism

```
# Threading (for I/O-bound tasks: network, file, database)
import threading

def download(url):
    # Simulate network request
    print(f"Downloading {url}...")
    time.sleep(2)
    print(f"Done: {url}")

threads = []
for url in ["url1", "url2", "url3"]:
```

```

    t = threading.Thread(target=download, args=(url,))
    threads.append(t)
    t.start()

for t in threads:
    t.join() # Wait for all threads to finish

# Multiprocessing (for CPU-bound tasks: math, image processing)
from multiprocessing import Pool

def process_chunk(data):
    return sum(x**2 for x in data)

with Pool(4) as pool: # 4 worker processes
    chunks = [range(0, 250000), range(250000, 500000),
               range(500000, 750000), range(750000, 1000000)]
    results = pool.map(process_chunk, chunks)
    total = sum(results)

# Async/Await (for high-concurrency I/O – web servers, API calls)
import asyncio

async def fetch_data(name, delay):
    print(f"Starting {name}")
    await asyncio.sleep(delay) # Non-blocking sleep
    print(f"Finished {name}")
    return f"Data from {name}"

async def main():
    # Run 3 tasks concurrently (not sequentially!)
    results = await asyncio.gather(
        fetch_data("API_1", 2),
        fetch_data("API_2", 1),
        fetch_data("API_3", 3),
    )
    print(results) # All three results

asyncio.run(main()) # Total time: ~3 seconds (not 6!)
```

When to Use What

Task Type	Tool	Example
I/O-bound (network, disk)	<code>threading</code> or <code>asyncio</code>	Web scraping, API calls
CPU-bound (computation)	<code>multiprocessing</code>	Image processing, ML training
High-concurrency I/O	<code>asyncio</code>	Web server handling 10,000 connections

Chapter 13: Regular Expressions

```

import re

text = "Contact us at support@example.com or sales@company.org"

# Find all email addresses
emails = re.findall(r'[\w.+-]+@[ \w-]+\.[\w.]+', text)
# ['support@example.com', 'sales@company.org']

# Common patterns
re.search(r'\d+', "abc123def")      # Match object for '123'
re.findall(r'\d+', "a1b2c3")        # ['1', '2', '3']
re.sub(r'\s+', ' ', "too many spaces") # "too many spaces"
re.split(r'[;,]', "a,b;c,d")        # ['a', 'b', 'c', 'd']

# Regex cheat sheet
# .      Any character except newline
# \d     Digit [0-9]
# \D     Non-digit
# \w     Word character [a-zA-Z0-9_]
# \W     Non-word character
# \s     Whitespace
# \S     Non-whitespace
# ^     Start of string
# $     End of string
# *     0 or more
# +     1 or more
# ?     0 or 1 (optional)
# {n}    Exactly n times
# {n,m}  Between n and m times
# [abc]  Character class: a, b, or c
# [^abc] NOT a, b, or c
# (...)  Capture group
# (?:...) Non-capturing group
# a|b    a or b

# Groups (extract parts of a match)
match = re.search(r'(\d{4})-(\d{2})-(\d{2})', "Date: 2026-10-03")
if match:
    year, month, day = match.groups()
    print(f"{day}/{month}/{year}") # "03/10/2026"

# Named groups
match = re.search(r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})', "2026-10-03")
print(match.group('year')) # "2026"

```

Chapter 14: Testing

```
# unittest (standard library)
import unittest

def factorial(n):
    if n < 0:
        raise ValueError("Negative input")
    if n <= 1:
        return 1
    return n * factorial(n - 1)

class TestFactorial(unittest.TestCase):
    def test_zero(self):
        self.assertEqual(factorial(0), 1)

    def test_positive(self):
        self.assertEqual(factorial(5), 120)
        self.assertEqual(factorial(1), 1)

    def test_negative(self):
        with self.assertRaises(ValueError):
            factorial(-1)

    def test_large(self):
        self.assertEqual(factorial(10), 3628800)

if __name__ == "__main__":
    unittest.main()

# Run: python -m unittest test_factorial.py -v

# pytest (third-party, more Pythonic – install with: pip install pytest)
# test_math.py
def test_addition():
    assert 1 + 1 == 2

def test_string():
    assert "hello".upper() == "HELLO"

def test_list():
    lst = [1, 2, 3]
    lst.append(4)
    assert lst == [1, 2, 3, 4]
    assert len(lst) == 4

# Run: pytest test_math.py -v
```

Chapter 15: Virtual Environments and Packages

```
# Create a virtual environment
python -m venv myenv

# Activate it
# Windows:
myenv\Scripts\activate
# Linux/Mac:
source myenv/bin/activate

# Install packages (into the virtual environment only)
pip install requests flask numpy pandas

# Save dependencies
pip freeze > requirements.txt

# Install from requirements file (on another machine)
pip install -r requirements.txt

# Deactivate
deactivate
```

PART IV: PYTHON FOR SPECIFIC DOMAINS

Chapter 16: Web Development (Flask Basics)

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, World!"

@app.route("/api/greet/<name>")
def greet(name):
    return jsonify({"message": f"Hello, {name}!", "status": "ok"})

@app.route("/api/add", methods=["POST"])
def add():
    data = request.get_json()
    result = data["a"] + data["b"]
    return jsonify({"result": result})

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

Chapter 17: Data Science Essentials

```
# NumPy – Fast numerical arrays
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
arr * 2          # [2, 4, 6, 8, 10] – element-wise!
np.mean(arr)     # 3.0
np.std(arr)      # 1.414...
np.dot(arr, arr) # 55 (dot product)

matrix = np.array([[1, 2], [3, 4]])
matrix @ matrix  # Matrix multiplication

np.zeros((3, 3)) # 3x3 matrix of zeros
np.ones((2, 4))  # 2x4 matrix of ones
np.random.randn(100) # 100 random numbers (normal distribution)

# Pandas – Data analysis
import pandas as pd

df = pd.DataFrame({
    "name": ["Alice", "Bob", "Charlie", "Diana"],
    "age": [25, 30, 35, 28],
    "salary": [50000, 60000, 70000, 55000]
})

df.head()          # First 5 rows
df.describe()      # Summary statistics
df[df["age"] > 28]  # Filter rows where age > 28
df.sort_values("salary", ascending=False)
df["salary"].mean() # Average salary
df.groupby("age")["salary"].mean() # Group by age, average salary
```

PART V: INTERVIEW PREPARATION

Chapter 18: Python Interview Questions — Easy

Q1: Reverse a String

```
def reverse_string(s):
    return s[::-1]

# Follow-up: Without slicing
def reverse_string_manual(s):
    chars = list(s)
    left, right = 0, len(chars) - 1
    while left < right:
```

```
    chars[left], chars[right] = chars[right], chars[left]
    left += 1
    right -= 1
    return ''.join(chars)
```

Q2: Check if a String is a Palindrome

```
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

# Test
is_palindrome("racecar")      # True
is_palindrome("A man a plan a canal Panama".replace(" ", "")) # True
```

Q3: FizzBuzz (Classic Screening Question)

```
def fizzbuzz(n):
    for i in range(1, n + 1):
        if i % 15 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)
```

Q4: Find Duplicates in a List

```
def find_duplicates(lst):
    seen = set()
    duplicates = set()
    for item in lst:
        if item in seen:
            duplicates.add(item)
        seen.add(item)
    return list(duplicates)

# One-liner using Counter
from collections import Counter
def find_duplicates(lst):
    return [item for item, count in Counter(lst).items() if count > 1]
```

Q5: Two Sum (LeetCode #1 — Most Common Interview Question)

```
def two_sum(nums, target):
    """Return indices of two numbers that add up to target."""
    seen = {} # value → index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

two_sum([2, 7, 11, 15], 9) # [0, 1] (because 2 + 7 = 9)
# Time: O(n), Space: O(n)
```

Q6: What is the difference between a list and a tuple?

Answer:

Feature	List	Tuple
Mutability	Mutable (can change)	Immutable (cannot change)
Syntax	[1, 2, 3]	(1, 2, 3)
Hashable	No (can't be dict key)	Yes (can be dict key)
Performance	Slightly slower	Slightly faster
Use case	Collection that changes	Fixed collection, dict keys, function returns

Q7: What does `*args` and `**kwargs` mean?

Answer: `*args` collects extra positional arguments into a **tuple**. `**kwargs` collects extra keyword arguments into a **dictionary**. They allow functions to accept any number of arguments.

```
def example(*args, **kwargs):
    print(args) # (1, 2, 3)
    print(kwargs) # {'name': 'Dhruv', 'age': 20}

example(1, 2, 3, name="Dhruv", age=20)
```

Chapter 19: Python Interview Questions — Medium

Q8: Implement a LRU Cache

```
from collections import OrderedDict

class LRUCache:
```

```

def __init__(self, capacity):
    self.cache = OrderedDict()
    self.capacity = capacity

def get(self, key):
    if key not in self.cache:
        return -1
    self.cache.move_to_end(key) # Mark as recently used
    return self.cache[key]

def put(self, key, value):
    if key in self.cache:
        self.cache.move_to_end(key)
    self.cache[key] = value
    if len(self.cache) > self.capacity:
        self.cache.popitem(last=False) # Remove least recently used

```

Q9: Merge Two Sorted Lists

```

def merge_sorted(l1, l2):
    result = []
    i = j = 0
    while i < len(l1) and j < len(l2):
        if l1[i] <= l2[j]:
            result.append(l1[i])
            i += 1
        else:
            result.append(l2[j])
            j += 1
    result.extend(l1[i:])
    result.extend(l2[j:])
    return result

```

Q10: Flatten a Nested List

```

def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item)) # Recursion!
        else:
            result.append(item)
    return result

flatten([1, [2, [3, 4], 5], [6, 7]]) # [1, 2, 3, 4, 5, 6, 7]

```

Q11: Group Anagrams

```

from collections import defaultdict

def group_anagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        key = ''.join(sorted(s)) # "eat" → "aet", "tea" → "aet"
        groups[key].append(s)
    return list(groups.values())

group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
# [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]

```

Q12: What is the GIL (Global Interpreter Lock)?

Answer: The GIL is a mutex that allows only one thread to execute Python bytecodes at a time, even on multi-core CPUs. This means:

- **Threading in Python does NOT achieve true parallelism for CPU-bound tasks.**
- Threading IS useful for I/O-bound tasks (waiting for network/disk), because the GIL is released during I/O operations.
- For CPU parallelism, use `multiprocessing` (separate processes, each with its own GIL) or write the CPU-heavy code in C/C++ (e.g., NumPy).

Q13: What are Python decorators? Explain with an example.

Answer: A decorator is a function that takes another function as input, adds some functionality, and returns a new function — without modifying the original. They use the `@decorator` syntax.

```

def require_auth(func):
    def wrapper(user, *args, **kwargs):
        if not user.get("authenticated"):
            raise PermissionError("Not authenticated")
        return func(user, *args, **kwargs)
    return wrapper

@require_auth
def get_secret_data(user):
    return "Top secret information"

# Equivalent to: get_secret_data = require_auth(get_secret_data)

```

Q14: Explain the difference between `deepcopy` and `shallow copy`.

```

import copy

original = [[1, 2, 3], [4, 5, 6]]

```

```
# Shallow copy: new outer list, but inner lists are SHARED
shallow = copy.copy(original)
shallow[0][0] = 99
print(original[0][0]) # 99! Inner list was shared.

# Deep copy: completely independent copy at ALL levels
original = [[1, 2, 3], [4, 5, 6]]
deep = copy.deepcopy(original)
deep[0][0] = 99
print(original[0][0]) # 1 (unchanged)
```

Q15: What is a generator and why use it?

Answer: A generator is a function that uses `yield` instead of `return`. It produces values lazily (one at a time), pausing between each yield. This means:

- **Memory efficient:** Can handle infinite sequences or huge datasets.
- **Lazy evaluation:** Values computed only when requested.
- **State preserved:** Local variables and execution position are saved between yields.

```
# Without generator: loads entire file into memory
lines = open("huge.log").readlines() # 10 GB file → 10 GB RAM!

# With generator: processes one line at a time
def read_lines(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

for line in read_lines("huge.log"): # Uses ~0 extra memory
    if "ERROR" in line:
        print(line)
```

Chapter 20: Python Interview Questions — Hard

Q16: Implement a Binary Search Tree

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, val):
```

```

        if not self.root:
            self.root = TreeNode(val)
        else:
            self._insert(self.root, val)

    def _insert(self, node, val):
        if val < node.val:
            if node.left is None:
                node.left = TreeNode(val)
            else:
                self._insert(node.left, val)
        else:
            if node.right is None:
                node.right = TreeNode(val)
            else:
                self._insert(node.right, val)

    def search(self, val):
        return self._search(self.root, val)

    def _search(self, node, val):
        if node is None:
            return False
        if val == node.val:
            return True
        elif val < node.val:
            return self._search(node.left, val)
        else:
            return self._search(node.right, val)

    def inorder(self):
        """Returns sorted elements."""
        result = []
        self._inorder(self.root, result)
        return result

    def _inorder(self, self, node, result):
        if node:
            self._inorder(node.left, result)
            result.append(node.val)
            self._inorder(node.right, result)

```

Q17: Valid Parentheses (LeetCode #20)

```

def is_valid(s):
    stack = []
    mapping = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in mapping:
            if not stack or stack[-1] != mapping[char]:

```

```

        return False
    stack.pop()
else:
    stack.append(char)

return len(stack) == 0

is_valid("([{}])") # True
is_valid("([])")   # False

```

Q18: Maximum Subarray Sum (Kadane's Algorithm)

```

def max_subarray(nums):
    max_sum = current_sum = nums[0]
    for num in nums[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum

max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]) # 6 (subarray [4,-1,2,1])

```

Q19: Longest Common Subsequence (Dynamic Programming)

```

def lcs(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]

lcs("abcde", "ace") # 3 ("ace")

```

Q20: Explain Python's Memory Management

Answer:

1. **Reference Counting:** Every object has a counter tracking how many variables point to it. When the count drops to 0, the object is immediately freed.
2. **Garbage Collector:** Handles circular references (A points to B, B points to A). The `gc` module detects and frees cycles.

3. **Memory Pooling:** Python pre-allocates memory for small objects (-5 to 256 for integers, short strings). This is why `a = 256; b = 256; a is b` returns `True`, but `a = 257; b = 257; a is b` may return `False`.
4. **Everything is an object:** Even `int`, `str`, and functions are heap-allocated objects. This is why Python uses more memory than C.

Q21: What is the difference between `is` and `==`?

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

a == b    # True   (same VALUE)
a is b    # False  (different OBJECTS in memory)
a is c    # True   (same OBJECT – c points to the exact same list)

# 'is' checks identity (are they the same object in memory?)
# '==' checks equality (do they have the same value?)

# Use 'is' only for: None, True, False
if x is None:      # CORRECT
if x == None:      # Works but not Pythonic
```

Chapter 21: Conceptual Interview Questions

Q22: What is the difference between `@staticmethod` and `@classmethod`?

Feature	@staticmethod	@classmethod
First parameter	None	<code>cls</code> (the class itself)
Access to class	No	Yes
Access to instance	No	No
Can modify class state	No	Yes
Use case	Utility function related to class	Factory method, alternative constructor

```
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day

    @classmethod
    def from_string(cls, date_str):
        year, month, day = map(int, date_str.split("-"))
        return cls(year, month, day) # Creates a Date instance
```

```

@staticmethod
def is_valid(date_str):
    parts = date_str.split("-")
    return len(parts) == 3 and all(p.isdigit() for p in parts)

d = Date.from_string("2026-10-03") # Uses classmethod
Date.is_valid("2026-10-03")      # Uses staticmethod, True

```

Q23: Explain Python's MRO (Method Resolution Order)

Answer: MRO determines the order in which Python looks up methods in a class hierarchy with multiple inheritance. Python uses the **C3 Linearization** algorithm.

```

class A:
    def method(self):
        return "A"

class B(A):
    def method(self):
        return "B"

class C(A):
    def method(self):
        return "C"

class D(B, C):
    pass

D().method() # "B" - Python searches: D → B → C → A
print(D.__mro__) # (D, B, C, A, object)

```

Q24: What are metaclasses?

Answer: A metaclass is the "class of a class." Just as an object is an instance of a class, a class is an instance of a metaclass. The default metaclass is `type`. Metaclasses allow you to customize class creation (add attributes, validate names, register classes, etc.).

```

# type is the metaclass of all classes
type(int)      # <class 'type'>
type(str)      # <class 'type'>

# Creating a class dynamically with type
MyClass = type('MyClass', (object,), {'x': 42, 'greet': lambda self: "hello"})
obj = MyClass()
obj.x         # 42
obj.greet()   # "hello"

```

Q25: Common Python Gotchas

```
# Gotcha 1: Mutable default arguments
def append_to(item, lst=[]): # DEFAULT LIST IS SHARED!
    lst.append(item)
    return lst

append_to(1) # [1]
append_to(2) # [1, 2] – NOT [2]! Same list object reused!

# Fix:
def append_to(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst

# Gotcha 2: Late binding closures
funcs = [lambda x: x + i for i in range(5)]
[f(0) for f in funcs] # [4, 4, 4, 4, 4] – NOT [0, 1, 2, 3, 4]!
# All lambdas capture the SAME variable i, which is 4 after the loop.

# Fix: default argument captures value at creation time
funcs = [lambda x, i=i: x + i for i in range(5)]
[f(0) for f in funcs] # [0, 1, 2, 3, 4]

# Gotcha 3: Integer caching
a = 256
b = 256
a is b # True (Python caches -5 to 256)

a = 257
b = 257
a is b # False (or True, depending on implementation)

# Gotcha 4: String concatenation in loops
# BAD (creates a new string object every iteration – O(n²))
result = ""
for s in big_list:
    result += s

# GOOD (O(n))
result = "".join(big_list)
```

Chapter 22: Coding Challenge Practice Guide

Difficulty Progression

Week	Focus	Problems per Day	Topics
1-2	Easy	2-3	Arrays, strings, hash maps, two pointers
3-4	Easy-Medium	2	Stacks, queues, linked lists, sorting
5-6	Medium	1-2	Binary search, sliding window, BFS/DFS
7-8	Medium-Hard	1	Trees, graphs, dynamic programming
9-10	Hard	1	DP optimization, tries, advanced graphs

The UMPIRE Method (For Solving Any Problem)

1. **Understand:** Read the problem. Ask clarifying questions. Identify inputs, outputs, edge cases.
2. **Match:** What pattern does this match? (Two pointers? Sliding window? BFS? DP?)
3. **Plan:** Write pseudocode. Talk through your approach BEFORE coding.
4. **Implement:** Translate pseudocode to clean Python.
5. **Review:** Walk through your code with a test case. Check edge cases.
6. **Evaluate:** Analyze time and space complexity. Can you optimize?

Common Patterns

Pattern	When to Use	Example Problems
Two Pointers	Sorted array, pair finding	Two Sum II, Container With Most Water
Sliding Window	Subarray/substring with constraint	Max subarray of size k, Longest substring without repeating
Hash Map	Frequency counting, lookup	Two Sum, Group Anagrams
Stack	Matching, nesting, undo	Valid Parentheses, Daily Temperatures
BFS	Shortest path, level-order	Binary tree level order, Shortest path in grid
DFS	Explore all paths, backtracking	Number of Islands, Permutations
Binary Search	Sorted data, find boundary	Search in rotated array, Find peak element
Dynamic Programming	Overlapping subproblems, optimal	Fibonacci, Coin Change, Longest Common Subsequence
Greedy	Local optimal = global optimal	Activity selection, Huffman coding

Essential LeetCode Problems (Top 25)

#	Problem	Pattern	Difficulty
1	Two Sum	Hash Map	Easy
20	Valid Parentheses	Stack	Easy
21	Merge Two Sorted Lists	Linked List	Easy

#	Problem	Pattern	Difficulty
53	Maximum Subarray	Kadane's / DP	Medium
70	Climbing Stairs	DP	Easy
121	Best Time to Buy and Sell Stock	Greedy	Easy
125	Valid Palindrome	Two Pointers	Easy
141	Linked List Cycle	Fast/Slow Pointer	Easy
206	Reverse Linked List	Linked List	Easy
217	Contains Duplicate	Hash Set	Easy
3	Longest Substring Without Repeating Characters	Sliding Window	Medium
15	3Sum	Two Pointers + Sort	Medium
33	Search in Rotated Sorted Array	Binary Search	Medium
49	Group Anagrams	Hash Map	Medium
56	Merge Intervals	Sorting	Medium
102	Binary Tree Level Order Traversal	BFS	Medium
200	Number of Islands	DFS/BFS	Medium
238	Product of Array Except Self	Prefix/Suffix	Medium
322	Coin Change	DP	Medium
146	LRU Cache	Hash Map + Linked List	Medium
5	Longest Palindromic Substring	DP / Expand	Medium
23	Merge k Sorted Lists	Heap / Divide & Conquer	Hard
76	Minimum Window Substring	Sliding Window	Hard
124	Binary Tree Maximum Path Sum	DFS	Hard
297	Serialize and Deserialize Binary Tree	BFS/DFS	Hard

Appendix A: Python Quick Reference

Built-in Functions (Most Used)

Function	Purpose	Example
<code>print()</code>	Output	<code>print("hello", end="")</code>
<code>input()</code>	Read user input	<code>name = input("Name: ")</code>
<code>len()</code>	Length	<code>len([1,2,3]) → 3</code>
<code>range()</code>	Number sequence	<code>range(5) → 0,1,2,3,4</code>

Function	Purpose	Example
<code>type()</code>	Check type	<code>type(42) → <class 'int'></code>
<code>isinstance()</code>	Type check	<code>isinstance(42, int) → True</code>
<code>int() / float() / str()</code>	Type conversion	<code>int("42") → 42</code>
<code>list() / tuple() / set() / dict()</code>	Create collection	<code>list("abc") → ['a','b','c']</code>
<code>sorted()</code>	Sort (returns new)	<code>sorted([3,1,2]) → [1,2,3]</code>
<code>reversed()</code>	Reverse iterator	<code>list(reversed([1,2,3])) → [3,2,1]</code>
<code>enumerate()</code>	Index + value	<code>for i,v in enumerate(lst)</code>
<code>zip()</code>	Pair sequences	<code>zip([1,2], ['a','b'])</code>
<code>map()</code>	Apply function	<code>list(map(str, [1,2,3]))</code>
<code>filter()</code>	Filter by condition	<code>list(filter(bool, [0,1,2]))</code>
<code>any() / all()</code>	Boolean aggregate	<code>any([False, True]) → True</code>
<code>min() / max() / sum()</code>	Aggregate	<code>sum([1,2,3]) → 6</code>
<code>abs()</code>	Absolute value	<code>abs(-5) → 5</code>
<code>round()</code>	Round number	<code>round(3.14159, 2) → 3.14</code>
<code>hash()</code>	Hash value	<code>hash("hello")</code>
<code>id()</code>	Memory address	<code>id(x)</code>
<code>dir()</code>	List attributes	<code>dir(str)</code>
<code>help()</code>	Documentation	<code>help(print)</code>
<code>vars()</code>	Object attributes	<code>vars(obj)</code>
<code>getattr()</code>	Get attribute by name	<code>getattr(obj, "name")</code>
<code>hasattr()</code>	Check attribute exists	<code>hasattr(obj, "name")</code>
<code>callable()</code>	Is it callable?	<code>callable(print) → True</code>
<code>open()</code>	Open file	<code>open("f.txt", "r")</code>
<code>exec()</code>	Execute string as code	<code>exec("x = 42")</code>
<code>eval()</code>	Evaluate expression	<code>eval("3 + 4") → 7</code>

String Methods Reference

Method	Purpose	Example
<code>.upper() / .lower()</code>	Case conversion	<code>"Hi".upper() → "HI"</code>
<code>.strip() / .lstrip() / .rstrip()</code>	Remove whitespace	<code>" hi ".strip() → "hi"</code>

Method	Purpose	Example
<code>.split(sep)</code>	Split into list	<code>"a,b,c".split(",") → ['a','b','c']</code>
<code>.join(lst)</code>	Join list into string	<code>", ".join(['a','b']) → "a,b"</code>
<code>.replace(old, new)</code>	Replace substring	<code>"hello".replace("l","r") → "herro"</code>
<code>.find(sub)</code>	Find index (-1 if not found)	<code>"hello".find("ll") → 2</code>
<code>.count(sub)</code>	Count occurrences	<code>"hello".count("l") → 2</code>
<code>.startswith()</code> / <code>.endswith()</code>	Prefix/suffix check	<code>"hello".startswith("he") → True</code>
<code>.isdigit()</code> / <code>.isalpha()</code> / <code>.isalnum()</code>	Character type check	<code>"42".isdigit() → True</code>
<code>.zfill(width)</code>	Zero-pad	<code>"42".zfill(5) → "00042"</code>
<code>.center(width)</code>	Center in field	<code>"hi".center(10) → " hi "</code>
<code>.format()</code>	String formatting	<code>"{} is {}".format("Pi", 3.14)</code>

You now have a comprehensive Python reference, from fundamentals through advanced features to interview preparation. The key to mastering Python — just like C — is deliberate practice. Solve one problem every day, and within a few months you'll be ready for any technical interview.